European Journal of Medicinal Chemistry 316 (2026) 119038

Contents lists available at ScienceDirect

European Journal of Medicinal Chemistry

journal homepage: www.elsevier.com/locate/ejmech

PyMACS: A python-based automation suite for GROMACS molecular dynamics setup, simulation, and analysis

Joseph M. Schulz [a], Robert C. Reynolds [b], Stephan C. Schürer [a,c,d,*]

[a] Department of Molecular and Cellular Pharmacology, University of Miami, Miami, FL, USA
[b] O'Neal Comprehensive Cancer Center, University of Alabama at Birmingham, Birmingham, AL, 35205, USA
[c] Sylvester Comprehensive Cancer Center, University of Miami Miller School of Medicine, Miami, FL, USA
[d] Frost Institute for Data Science & Computing, University of Miami, Miami, FL, USA

A B S T R A C T

Molecular-dynamics (MD) simulations provide atom-level insight into biomolecular motion and are widely used to complement structure-guided drug discovery, ligand optimization, and mechanistic structural biology. However, reproducible MD remains technically challenging because users must coordinate structure cleanup, force-field-compatible topology generation, ligand parameterization, solvation, ionization, energy minimization, equilibration, production simulation, trajectory processing, and downstream interpretation; errors at any stage can compromise results and discourage non-expert users. Here, we present PyMACS, an open-source Python automation framework for GROMACS-based molecular dynamics using the CHARMM36 or CHARMM36/LJ-PME biomolecular force fields together with CGenFF-compatible small-molecule parameterization. PyMACS accepts common structural input formats, including PDB, CIF, and mmCIF files, and is designed to operate on a broad range of user-supplied biomolecular systems rather than being restricted to a single target class or experimentally resolved protein-ligand complexes. Compatible inputs include experimental structures, docked or model-derived complexes, computationally predicted biomolecular assemblies, apo proteins, protein-ligand systems, cofactor-containing complexes, peptide-protein assemblies, protein-protein complexes, RNA-, DNA-, and mixed nucleic-acid-protein systems for setup and simulation workflows when residue naming and CHARMM/GROMACS topology generation support the input, and PROTAC or other ternary-complex models when force-field compatibility requirements are met. The pipeline automates structure preparation, ligand and component handling, GROMACS topology generation, system assembly, solvation, ionization, equilibration, production MD, checkpoint-aware execution, and post-simulation analysis. PyMACS also converts completed trajectories into interpretable outputs including RMSD, RMSF, radius of gyration, ligand-stability metrics, residue-contact summaries, interaction-network visualizations, CSV exports, and report-ready figures. Designed for both interactive novice use and headless reproducible execution, PyMACS lowers the barrier to rigorous MD simulation while preserving configurable control over scientific parameters, computational resources, and analysis thresholds. By integrating setup, simulation, analysis, and visualization into a transparent workflow, PyMACS enables medicinal chemists and structural biology researchers to evaluate candidate binding hypotheses, compare biomolecular systems, and generate reproducible MD-derived evidence with substantially reduced manual overhead.

1. Introduction

1.1. Molecular-dynamics workflow

Molecular dynamics (MD) simulations have become a foundational tool across structural biology, biophysics, and drug discovery, providing atom-level insight into biomolecular structure, dynamics, and interactions beyond what static experimental structures can capture. [1–4]. MD has been widely applied to study protein folding and conformational change, enzyme catalysis, protein-protein interactions, and the binding behavior of small molecules, peptides, and larger heterobifunctional compounds. [5]. In the context of modern medicinal chemistry, MD simulations play an increasingly important role in characterizing ligand binding stability, induced fit, and allosteric effects, and have been extended to more complex modalities such as proteolysis-targeting chimeras (PROTACs), where ternary complex formation and dynamic protein-protein interfaces are central to mechanism of action. [6]. Across these diverse applications, MD provides a unifying framework for connecting molecular structure to function by explicitly modeling time-dependent atomic motion under physically grounded force fields. [7].

MD simulations propagate Newton's equations of motion for classical systems. [4]. As illustrated in Fig. 1, typical workflow includes system preparation & parameterization (cleaning the structure, generating a

This article is part of a special issue entitled: Comp small drug design published in European Journal of Medicinal Chemistry.

* Corresponding author. Department of Molecular and Cellular Pharmacology, University of Miami, Miami, FL, USA.
E-mail address: sschurer@miami.edu (S.C. Schürer).

https://doi.org/10.1016/j.ejmech.2026.119038
Received 7 April 2026; Received in revised form 28 May 2026; Accepted 3 June 2026
Available online 3 June 2026
0223-5234/© 2026 The Authors. Published by Elsevier Masson SAS. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

force-field-compatible topology, adding solvent and ions), energy minimization to remove steric clashes, equilibration under constant-temperature and constant-pressure ensembles to stabilize thermodynamic variables, and a production phase where trajectories are collected for analysis. [8]. Common analysis metrics include root-mean-square deviation (RMSD), root-mean-square fluctuation (RMSF) and radius of gyration (Rg); RMSD quantifies conformational changes, RMSF measures per-residue flexibility and the Rg reflects molecular compactness. [9].

1.2. Force fields and small-molecule parameterization

Accurate MD results hinge on the underlying force field. The CHARMM additive force-field family covers biomolecules such as proteins, nucleic acids including RNA, carbohydrates, lipids, and monatomic ions, while the associated CHARMM General Force Field (CGenFF) provides parameters for drug-like molecules. [10–15]. CGenFF encompasses a wide variety of chemical groups, including heterocyclic scaffolds, and emphasizes parameter quality over transferability. Its automated assignment workflow includes bond perception, atom typing, bonded-parameter assignment, partial-charge assignment, and penalty scores that help users judge whether additional parameter optimization may be needed.

When running CHARMM36 systems in GROMACS, typical MD settings include constraining bonds involving hydrogen atoms with the LINCS algorithm, applying particle-mesh Ewald electrostatics, and equilibrating sequentially under NVT and NPT ensembles with appropriate thermostat and barostat coupling. For standard CHARMM36 workflows, Lennard-Jones interactions are commonly treated with force-switching cutoffs in the approximately 10-12 Å range. [15]. For CHARMM36/LJ-PME workflows, the long-range Lennard-Jones particle-mesh-Ewald treatment permits use of a 9 Å nonbonded cutoff, improving computational efficiency while maintaining consistency with the LJ-PME parameterization. [14]. The LJ-PME implementation supports the CHARMM force-field family broadly; however, specific lipid LJ-PME parameter optimization was required because lipid bilayers are particularly sensitive to Lennard-Jones truncation. The charmm2gmx utility provides an automated and validated method to port the CHARMM additive force field to GROMACS; it reads upstream CHARMM data and produces self-documenting GROMACS files without hard-coded parameters. [16].

1.3. Challenges and need for automation

While tutorials exist for performing MD simulations, the numerous manual steps create barriers for non-experts. [3,8,17–23]. Users must clean PDB files, assign protonation states, generate topology files for proteins and ligands, merge coordinates, solvate and ionize the system, choose appropriate MD parameters and run several GROMACS commands in the correct order. [8]. Errors during any stage, such as missing atoms or mis-typed ligands, propagate into the simulation. Even experienced users can waste time re-running simulations due to trivial mistakes. An automated pipeline that encapsulates best practices, provides sensible defaults and records decisions for reproducibility would make MD accessible to a broader audience, particularly medicinal chemists who may not be trained computational chemists.

2. Implementation and workflow

The PyMACS workflow is organized as a modular sequence of scripts that guide the user from initial structure preparation through molecular dynamics simulation, trajectory analysis, and report generation. Table 1 summarizes the major workflow phases, the corresponding computational steps, and the scripts responsible for each stage.

While Table 1 provides the overall workflow structure, PyMACS is not limited to a fixed, one-size-fits-all protocol. The pipeline provides novice-accessible defaults while exposing key setup, simulation, and analysis parameters through command-line arguments and external template files. This design allows users to modify scientific conditions, computational resources, and analysis thresholds without editing the core Python scripts. Table 2 summarizes the principal default behaviors and user-configurable options available across the workflow.

2.1. Preparation and parameterization (script 1)

The first stage of the workflow is handled by *1_AutomateGromacs.py*, which performs structure preparation, chain assignment, ligand parameterization when applicable, and force-field setup. The script can be executed either interactively, through guided user prompts, or in a fully headless mode using command-line arguments, enabling both exploratory and high-throughput use cases. The starting input is a user-supplied PDB structure representing the system to be simulated, including experimentally resolved co-crystal complexes, docked protein-ligand complexes, modeled binding poses, peptide-protein assemblies, protein-protein complexes, or apo protein structures. PyMACS assumes that any desired bound pose has already been generated upstream and does not itself perform docking or pose prediction.

PyMACS accepts PDB, CIF, and mmCIF coordinate files. CIF/mmCIF inputs are converted internally into a PDB-style setup intermediate before validation, cleanup, and topology generation. RCSB-style formatting is recommended for PDB inputs, but the workflow does not require that every file originate directly from the RCSB PDB, provided residue names, chain identifiers, atom coordinates, and ligand residue names can be interpreted consistently. The setup script performs preflight validation on the prepared PDB-style intermediate and can optionally remove waters, ions, common solvents/buffers, or user-specified residue names before setup.

RNA-containing or other nucleic-acid-containing systems can be prepared when compatible CHARMM/GROMACS residue naming and topology-generation support are available. In the current implementation, nucleic-acid residues are excluded from ligand autodetection so that RNA is not mistakenly treated as a small-molecule ligand. The supplementary materials now include representative RNA-containing setup and biological-mode simulation examples. These examples demonstrate that mixed RNA-protein systems can proceed through preparation, solvation, ionization, equilibration, and production MD

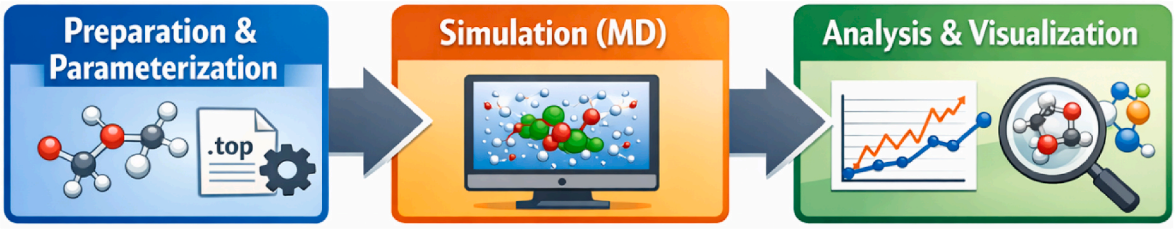


Fig. 1. Schematic overview of a typical molecular dynamic simulation workflow. The process includes preparation and parameterization of the system, simulation using MD engines to evolve atomic positions over time, and analysis/visualization of trajectories to extract structural and dynamical information.

Table 1
Overview of the PyMACS automated molecular dynamics pipeline. The PyMACS workflow is organized into three major phases: system preparation, simulation execution, and post-simulation analysis and reporting. The simulation-execution phase itself comprises four sequential molecular dynamics sub-stages: energy minimization, NVT equilibration, NPT equilibration, and production MD.

Phase	Step	Description	Script
System preparation	Structure cleaning	Read PDB, fix missing residues/atoms, assign chains	1_AutomateGromacs.py
System preparation	Parameterization	Generate ligand parameters with CGenFF; convert to GROMACS using charmm2gmx	1_AutomateGromacs.py
System preparation	Topology generation	Run pdb2gmx to generate protein topology and, when applicable, nucleic-acid or mixed biomolecular topology with CHARMM36/LJ-PME; merge protein and ligand when applicable	1_AutomateGromacs.py
System preparation	Solvation and ionization	Solvate the system and add ions to neutralize at 0.15 M	1_AutomateGromacs.py
Simulation execution	Energy minimization	Steepest-descents minimization to relax the system	2_AutomateGromacs.py
Simulation execution	NVT equilibration	Constant-volume and temperature equilibration with position restraints	2_AutomateGromacs.py
Simulation execution	NPT equilibration	Constant-pressure equilibration with barostat and gradual release of restraints	2_AutomateGromacs.py
Simulation execution	Production MD	Final unrestrained MD run for user-defined time; outputs trajectory and energies	2_AutomateGromacs.py
Analysis and reporting	Trajectory analysis	Compute RMSD, RMSF, Rg, hydrogen bonds, contacts, and visualization outputs	3A_AutomateGromacs.py, 3B_NETWORX.py
Analysis and reporting	Figure compilation	Assemble plots and summaries into a PDF report using 4_MDfigs.txt	4PDF4MD.py

when residue naming and CHARMM/GROMACS topology generation are compatible. More broadly, PyMACS is intended for compatible user-supplied biomolecular systems, including RNA, DNA, mixed protein-nucleic-acid assemblies, apo proteins, protein-protein complexes, ligand-bound systems, cofactor-containing systems, and related preassembled complexes, rather than being restricted to protein-ligand systems alone. These RNA-containing examples are intended as representative setup and production-MD demonstrations; they should not be interpreted as a systematic RNA-ligand benchmark or as a claim that all RNA target classes are automatically supported.

Users begin by specifying the input PDB file and defining chain identities. Chain names can be provided explicitly using the –chain-map argument (for example, "A:Kinase,B:Inhibitor") or via simplified chain-naming options. To ensure consistency across repeated runs, chain assignments are cached locally in a.chainmap.json file, allowing automated reuse without additional user input. For apo proteins or biomolecular assemblies that do not require separate small-molecule parameterization, the workflow proceeds directly from structure preparation to topology generation using the supplied chain definitions.

Prior to topology generation, the script performs structural cleanup to address common issues in experimental or predicted structures. Missing residues, atoms, and hydrogens are added using PDBFixer/OpenMM, with protonation states assigned at a user-defined pH. [24]. Missing heavy atoms and side-chain atoms are rebuilt from residue templates and local stereochemical context; therefore, users should inspect repaired regions, especially unresolved loops or chemically unusual environments, before interpreting downstream MD results. This correction step is critical for successful downstream setup, as incomplete structures can lead to topology-generation failures or instability during subsequent energy minimization in GROMACS.

For systems containing a non-protein small molecule, cofactor, or other parameterized ligand-like component, the user specifies the ligand residue name present in the input structure. The ligand may originate from a co-crystal structure, a docking workflow, or another modeled complex, provided that its coordinates are already present in the starting PDB. The script automatically extracts the ligand coordinates, converts them into MOL2 format, and submits them to CGenFF for parameterization. CGenFF assigns atom types and partial charges using a rule-based atom-typing framework combined with an extended bond-charge increment scheme. [13]. The resulting CHARMM stream (.str) file is then converted into GROMACS-compatible.itp and.prm files using cgenff_charmm2gmx_py3_nx2.py, a Python wrapper around the charmm2gmx conversion utility. When multiple non-protein components are present, they should be uniquely named in the input structure so that they can be identified and processed unambiguously.

The default automated ligand workflow is intended for non-covalently bound ligands, cofactors, or ligand-like components that can be parameterized independently and then merged with the receptor coordinates. For likely covalent ligands, PyMACS detects LINK records and ligand-related CONECT records during PDB preflight and warns or stops by default unless an expert override is supplied. Covalently bound systems generally require user-supplied or manually curated topology definitions to ensure that the protein-ligand bond, bonded terms, and atom typing are represented consistently. [12,13,25–30].

Default molecular dynamics settings, including constrained hydrogen-bond lengths, nonbonded interaction settings, thermostat/barostat choices, and step counts, are defined in external.mdp template files rather than being hard-coded into the Python scripts. The workflow supports user-provided MDP directories and per-stage MDP overrides, allowing advanced users to modify integrators, thermostats, barostats, simulated annealing schedules, restraint protocols, and production lengths while preserving the default templates for novice users. [31,32].

The assembled system is then placed in an SPC water box and neutralized by the addition of counter-ions, with NaCl added to a default physiological concentration of 0.15 M. The default setup remains a cubic box with a 1.0 nm solute-to-box-edge buffer for backward compatibility, but users can explicitly select the box geometry and buffer distance through –box-type and –box-distance. Supported GROMACS-compatible options include cubic, dodecahedron, and octahedron boxes, allowing users to reduce solvent overhead when appropriate. [31,32].

At completion (Fig. 2), the script produces a fully prepared simulation directory containing a master topology file (topol.top) with all force-field and ligand includes, an index file mapping chains to atom ranges, and a ready-to-run energy minimization input (em.tpr). All GROMACS and external commands executed during preparation are logged to mdrun.log, ensuring full transparency and reproducibility of the setup process. In its current implementation, PyMACS focuses on automated MD preparation and simulation of user-supplied complexes rather than upstream generation or enumeration of alternative poses, ionization states, or tautomeric forms.

Key configuration choices are written to the run log, including PDB preflight results, selected box geometry, MDP template provenance, and other setup-stage decisions. This makes the preparation stage easier to audit and reproduce across systems.

Table 2
Default and user-configurable PyMACS settings. PyMACS provides novice-accessible defaults while exposing setup, simulation, and analysis parameters for advanced users. Configuration is performed through command-line arguments and/or external template files, allowing users to modify scientific conditions without editing the core Python scripts.

Workflow component	Default behavior	User-configurable control	Script/stage	Purpose
Input structure validation	Performs preflight checks for ATOM/HETATM records, coordinate parsing, residue names, chain IDs, ligand presence, and ambiguous ligand-like residues; nucleic-acid residues are excluded from small-molecule ligand autodetection	–strict-pdb-validation	1_AutomateGromacs.py/ setup	Helps identify malformed or ambiguous PDB inputs before topology generation
Chain naming	Detects chains and caches names for reuse in. chainmap.json	–chain-map; –chain-names	1_AutomateGromacs.py/ setup	Enables reproducible chain labeling across repeated runs
Ligand workflow selection	Treats supplied ligand residue as a non-covalent ligand-like component	–ligand; –mode ligand; –mode protein	1_AutomateGromacs.py/ setup	Distinguishes protein-only/ biomolecule-only and ligand-containing workflows
Covalent-ligand handling	Stops or warns when LINK/CONECT records suggest covalent ligand attachment	–allow-covalent-ligand	1_AutomateGromacs.py/ setup	Prevents inappropriate use of the default non-covalent ligand workflow for covalently bound systems
Simulation box geometry	Cubic box with 1.0 nm solute-to-box-edge distance	–box-type; –box-distance	1_AutomateGromacs.py/ setup	Allows users to change box shape and solvent padding
Setup-stage MDP files	Uses default setup templates such as ions.mdp and em.mdp	–mdp-dir; –ions-mdp; –em-mdp	1_AutomateGromacs.py/ setup	Allows replacement of setup-stage GROMACS parameter files
Simulation-stage MDP files	Uses default em.mdp, nvt.mdp, npt.mdp, and md.mdp templates	–mdp-dir; –em-mdp; –nvt-mdp; –npt-mdp; –md-mdp	2_AutomateGromacs.py/ simulation	Allows users to modify integrators, thermostats, barostats, restraints, step counts, simulated annealing, or production length through external MDP files
Equilibration protocol	Runs the standard sequence of energy minimization, NVT equilibration, NPT equilibration, and production MD	–equilibration-plan	2_AutomateGromacs.py/ simulation	Allows advanced users to define multi-stage equilibration protocols through a JSON plan
Index groups	Uses automatically generated GROMACS index groups	–index-file	2_AutomateGromacs.py/ simulation	Allows custom atom selections or user-defined index groups
CPU/GPU resources	Uses automatic or default CPU/GPU behavior depending on available hardware and selected compute mode	–compute; –gpu; –gpu-id; –gpu-ids; –no-gpu; –ntomp; –ntmpi; –threads	2_AutomateGromacs.py/ simulation	Allows users to control CPU threads, MPI ranks, GPU selection, multi-GPU strings, or CPU-only execution
Production restart behavior	Runs the standard full workflow unless restart files are detected and restart mode is requested	–production_only; –resume; –force_restart; –ns	2_AutomateGromacs.py/ simulation	Supports checkpoint-safe production continuation or clean restart
Analysis mode	Detects or prompts for ligand, protein-only/ biomolecule-only, peptide/protein-protein, or PROTAC-style analysis	–mode; –ligand; –headless	3A_AutomateGromacs.py/ analysis	Routes the trajectory through the appropriate analysis workflow
Binding-pocket definition	Uses residues within 5.0 Å of the ligand or partner molecule	–pocket-cutoff	3A_AutomateGromacs.py; 3_PROTAC_Analysis.py/ analysis	Allows users to widen or narrow the binding-pocket definition
Contact definition	Reports ligand-protein or partner-protein contacts using a default 4.0 Å cutoff	–contact_cutoff; –distance-cutoff	3A_AutomateGromacs.py; 3_PROTAC_Analysis.py/ analysis	Allows tuning of residue-contact persistence calculations
Hydrogen-bond-like classification	Uses a default 3.5 Å distance cutoff and documents a 135° angle criterion	–hbond-distance-cutoff; –hbond-angle-cutoff	3A_AutomateGromacs.py/ analysis	Separates narrower hydrogen-bond-like events from broader distance-based contacts
Contact-frequency filtering	Uses default minimum contact-fraction thresholds for residue inclusion in plots	–min_contact_frac; –minfrac; PROTAC-specific contact-map/network thresholds	3A_AutomateGromacs.py; 3_PROTAC_Analysis.py/ analysis	Allows users to control which persistent contacts are visualized
PROTAC/ternary-complex sampling	Uses frame-stride defaults for computationally intensive ternary-complex analyses	–frame-step; –quick-test; –basic-only; –contacts-only; –start-frame; –end-frame	3_PROTAC_Analysis.py/ PROTAC analysis	Allows rapid testing, reduced analysis, or full frame-by-frame PROTAC analysis
Output reporting	Writes standard plots, interaction summaries, and figurebook-ready outputs	–outdir; –net-out; 4_MDfigs. txt	3A_AutomateGromacs.py; 3B_NETWORX.py; 4PDF4MD.py	Allows output organization and figurebook customization

Script 1 (1_AutomateGromacs.py) automates structure cleaning, ligand parameterization, force-field setup, and system assembly prior to molecular dynamics simulation. Input protein-ligand or other compatible biomolecular structures are repaired and protonated, ligands are parameterized using CGenFF and converted for GROMACS when applicable, and protein/nucleic-acid topologies are generated with CHARMM36 or CHARMM36/LJ-PME. The resulting complex is solvated, neutralized, and prepared for energy minimization, producing a fully reproducible, simulation-ready system.

2.2. Equilibration and production MD (script 2)

The second stage of the workflow is performed by *2_AutomateGromacs.py*, which takes the fully prepared system from the setup step and executes the molecular dynamics simulation through a series of standardized stages (Fig. 3). These stages are designed to ensure numerical stability, thermodynamic equilibration, and reproducible production trajectories suitable for downstream analysis. In this manuscript, "production simulation" refers specifically to the final unrestrained trajectory-generation stage, whereas energy minimization and NVT/NPT equilibration are treated as preceding simulation sub-stages.

The simulation begins with energy minimization, during which steric clashes and unfavorable contacts introduced during system assembly are relaxed. The script uses gmx grompp to compile the minimization input and gmx mdrun to perform a steepest-descents minimization. Convergence criteria, step limits, and force tolerances are defined in the

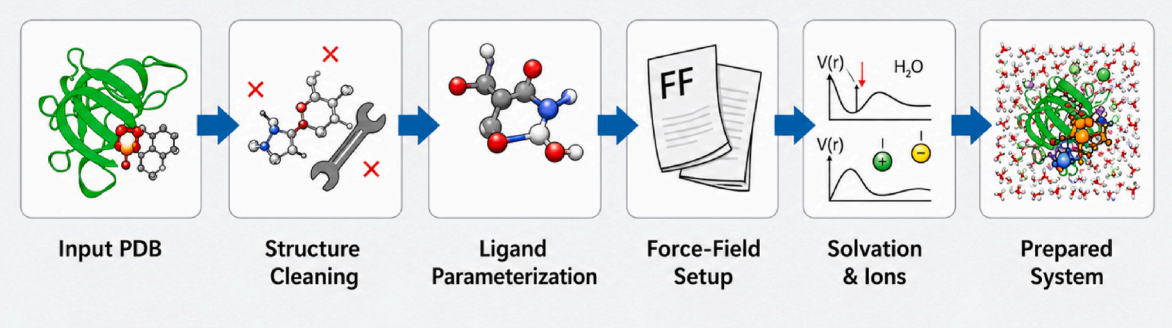


Fig. 2. Overview of system preparation and parameterization in PyMACS. Script 1 (1_AutomateGromacs.py) automates structure cleaning, ligand parameterization, force-field setup, and system assembly prior to molecular dynamics simulation. Input protein-ligand structures are repaired and protonated, ligands are parameterized using CGenFF and converted for GROMACS, and protein topologies are generated with CHARMM36 or CHARMM36/LJ-PME. The resulting complex is solvated, neutralized, and prepared for energy minimization, producing a fully reproducible, simulation-ready system.

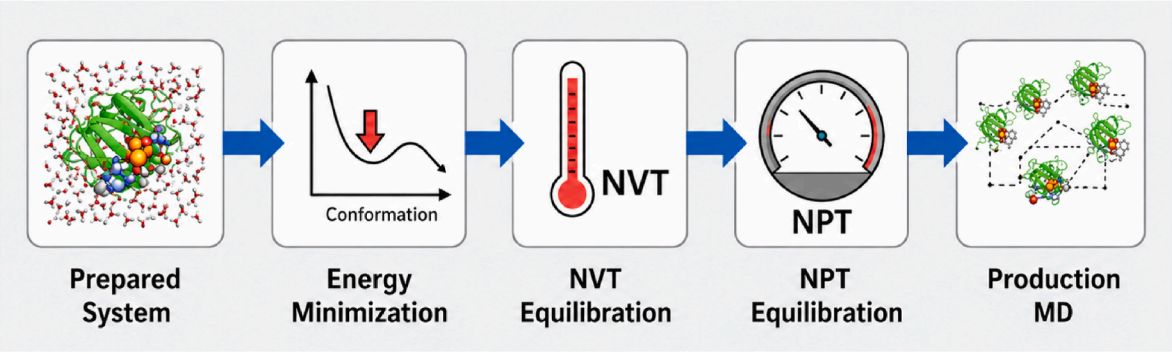


Fig. 3. Overview of equilibration and production molecular dynamics in PyMACS. Script 2 (2_AutomateGromacs.py) executes a standardized molecular dynamics protocol consisting of energy minimization, NVT equilibration, NPT equilibration, and production simulation. Energy minimization removes unfavorable contacts, followed by temperature and pressure equilibration under positional restraints to ensure numerical stability and correct system density. The equilibrated system is then propagated in an unrestrained production run, generating trajectories, energies, and checkpoint files for downstream analysis and restartability.

provided em.mdp template, ensuring consistent behavior across systems.

The script resolves and validates the MDP file used for each stage before execution. Users may use the bundled defaults or provide custom files through –mdp-dir, –em-mdp, –nvt-mdp, –npt-mdp, and –md-mdp. This design keeps the novice workflow simple while allowing expert users to alter thermostat/barostat protocols, simulated annealing, integrators, restraint schedules, and step counts without editing Python code.

Following minimization, the system undergoes NVT equilibration at constant volume and temperature. Initial atomic velocities are assigned from a Maxwell-Boltzmann distribution corresponding to the target temperature. Temperature coupling is applied using standard thermostats such as velocity-rescaling or Nosé-Hoover, as specified in the NVT parameter file. During this phase, positional restraints are typically applied to heavy atoms of the protein and ligand to prevent large-scale structural drift while the solvent and ions equilibrate around the fixed solute.

For workflows requiring finer equilibration control, PyMACS supports an optional JSON equilibration plan that can define an ordered series of NVT/NPT or other equilibration sub-stages, each with its own MDP file. When no plan is provided, the original NVT followed by NPT workflow is preserved.

Once thermal equilibrium has been achieved, the simulation proceeds to NPT equilibration, in which pressure coupling is introduced to allow the simulation box volume to fluctuate and reach the correct density. A barostat such as Parrinello-Rahman is used to regulate pressure, and positional restraints are gradually weakened or released according to the simulation protocol. During this stage, the script monitors key thermodynamic quantities, including potential energy, temperature, and pressure, to ensure stable equilibration before entering production.

The final stage is the production molecular dynamics run, in which the equilibrated system is propagated for the user-specified simulation time, typically ranging from tens to hundreds of nanoseconds. Trajectories are written to compressed.xtc files, while energies and checkpoints are stored in.edr and.cpt files, respectively. The inclusion of checkpoint files allows simulations to be safely restarted in the event of interruption without loss of progress.

Throughout execution, the script automatically selects an appropriate number of OpenMP threads based on available CPU cores and detects GPU resources when GROMACS has been compiled with CUDA support. At each stage of the workflow, return codes from GROMACS commands are checked explicitly; any failure results in immediate termination with informative diagnostic output, facilitating rapid troubleshooting and robust batch execution.

Resource allocation is explicit and user-configurable. Users can specify OpenMP threads, MPI ranks, GPU identifiers, multi-GPU strings, or CPU-only execution using –ntomp, –ntmpi, –gpu-id/–gpu-ids, and –no-gpu. The exact GROMACS mdrun command used for each stage is logged. PyMACS also preserves automatic/default behavior when these options are omitted. In addition, users may supply a custom GROMACS index file through –index-file; otherwise, the workflow retains automatic index generation.

2.3. *Trajectory analysis and visualization (scripts 3A, 3B, and 4)*

The final stage of the PyMACS pipeline performs post-simulation analysis and visualization of the completed molecular dynamics

trajectory (Fig. 4). This stage is implemented through 3A_Automate-Gromacs.py, which carries out structural and interaction-based analyses, 3B_NETWORX.py, which constructs journal-grade ligand-protein interaction diagrams, and 4PDF4MD.py, which assembles outputs into a unified report. Although PyMACS can be used to prepare and simulate a broad range of biomolecular systems, including apo proteins, peptide-protein assemblies, protein-protein complexes, RNA-containing systems, DNA-containing systems, and mixed protein-nucleic-acid assemblies, the current analysis layer is most fully developed for ligand-containing systems, particularly protein-ligand and related ternary-complex workflows. For nucleic-acid-containing systems, the implemented support is strongest at the setup and simulation-execution level; RNA/DNA-specific ligand-design analysis and broad RNA-ligand benchmarking remain outside the present validation scope.

Trajectory analysis is primarily performed using MDAnalysis and MDTraj, with optional GPU acceleration via CuPy for distance-based calculations. [33–35]. The workflow begins by generating a recentered subset trajectory (Final_Trajectory.xtc/.pdb) and, for ligand-containing systems, an automatically refined binding-pocket trajectory (binding_pocket_only.xtc/.pdb) defined by protein residues within a user-configurable cutoff of the ligand across the simulation. The default cutoff remains 5.0 Å for backward compatibility, but users can now modify this value through –pocket-cutoff. These reduced trajectory files support both downstream automated analysis and direct structural inspection in external molecular-graphics tools. [33–35].

A core set of general structural analyses is applicable across many simulation types. These include global protein backbone RMSD, full-complex RMSD, chain-aware RMSF, Rg, hydrogen-bond statistics, and, when available, secondary-structure persistence analysis. Together, these metrics provide a quantitative view of conformational drift, flexibility, compactness, and structural stabilization over time, and remain informative even in systems where no small-molecule ligand is present.

For ligand-containing systems, PyMACS provides an additional ligand-centric analysis layer optimized for small molecules and related modalities. These analyses include ligand RMSD and per-atom RMSF, dynamic binding-pocket extraction, frame-wise ligand-protein contact mapping, residue contact-frequency plots, and classification of persistent interaction types such as hydrogen bonds, hydrophobic contacts, ionic interactions, and water-mediated interactions. These outputs are intended to provide a chemically interpretable description of binding stability and residue-level interaction persistence across the trajectory.

Interaction reporting distinguishes broad distance-based contacts from hydrogen-bond-like interactions. Contacts are treated as proximity events within a configurable distance cutoff, whereas hydrogen-bond-like events are reported separately because they represent a narrower interaction class requiring donor/acceptor chemistry and, where available, distance/angle criteria. The analysis interface exposes contact and hydrogen-bond-related cutoffs through –contact_cutoff, –hbond-distance-cutoff, and –hbond-angle-cutoff, while preserving previous defaults. Hydrophobic and ionic classifications are reported as chemically interpretable residue-level summaries and should be interpreted as screening-level interaction annotations rather than substitute free-energy estimates.

Interaction data can then be passed to 3B_NETWORX.py, which generates publication-ready ligand-protein interaction diagrams. In these visualizations, ligand atoms are rendered from chemically accurate 2D layouts derived from CGenFF MOL2 files, while interacting residues are arranged around the ligand and connected by interaction-type-specific edges. The resulting figures provide an intuitive summary of binding-site organization, interaction persistence, and residue chemistry.

Finally, 4PDF4MD.py compiles the generated plots, contact summaries, and representative structural snapshots into a multi-panel PDF report guided by 4_MDfigs.txt. Depending on the simulated system, the report may include global stability metrics, flexibility analyses, ligand-centered interaction plots, and network diagrams, enabling rapid generation of figures suitable for internal review, supplementary materials, or comparative studies. Optional, user-modifiable companion scripts can further extend this analysis framework to specialized applications such as PROTAC or ternary-complex evaluation.

The repository includes an output-gallery document describing expected analysis products, including protein RMSD, RMSF, Rg, ligand RMSD/RMSF, binding-pocket contact heatmaps, residue contact-frequency plots, interaction composition plots, NETWORX panels, and the final MD_ANALYSIS_FIGUREBOOK.pdf.

Representative outputs generated from a single ligand-bound PyMACS analysis run are shown in Fig. 5, including structural-stability metrics, ligand-centered dynamics, residue-contact persistence, binding-importance ranking, and NETWORX interaction-network visualization.

Together, these analysis and visualization steps transform raw MD trajectories into a rich, quantitative, and visually interpretable dataset, enabling rigorous assessment of structural stability, conformational dynamics, and, where applicable, ligand-binding and interaction persistence across a broad range of biomolecular simulations.

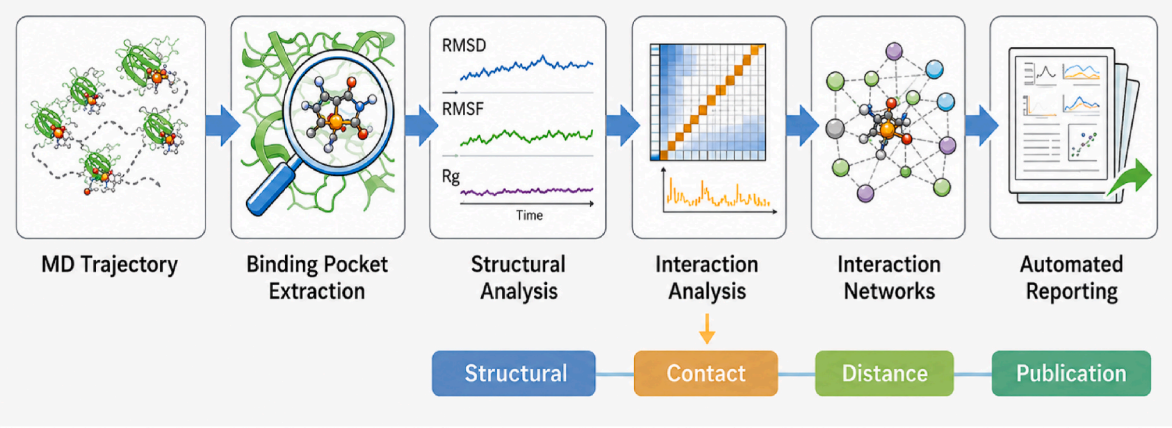


Fig. 4. Overview of trajectory analysis and visualization in PyMACS. Scripts 3A, 3B, and 4 perform automated post-simulation analysis of molecular dynamics trajectories. The workflow includes trajectory recentering, computation of general structural stability and flexibility metrics, and, for ligand-containing systems, binding-pocket extraction, ligand-protein contact analysis, and interaction-network generation, followed by automated compilation of publication-ready figures into a unified report.

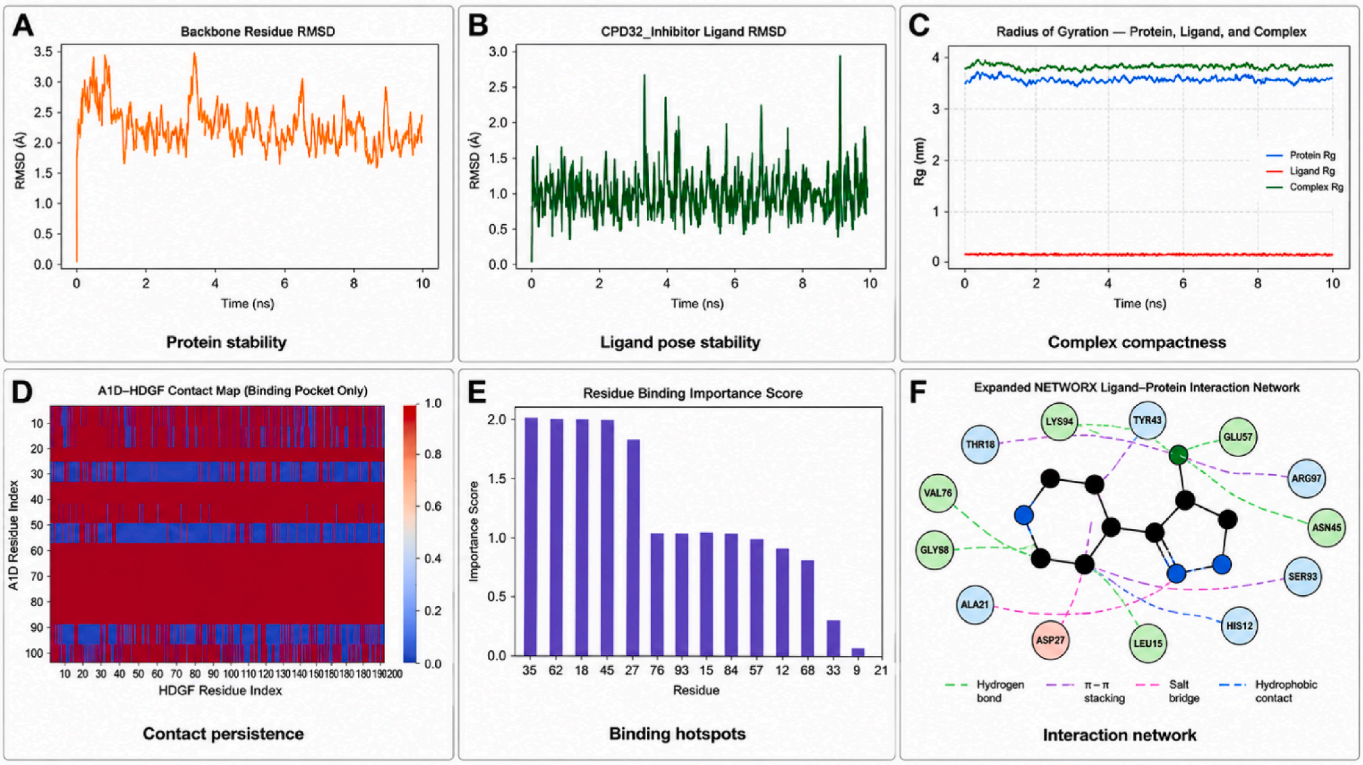


Fig. 5. Representative outputs from a single PyMACS ligand-bound molecular dynamics analysis run. PyMACS converts a completed GROMACS trajectory into structural, dynamical, and interaction-level outputs suitable for interpretation and reporting. (A) Global protein backbone RMSD monitors overall structural stability across the trajectory. (B) Ligand RMSD tracks whether the bound pose remains stable within the binding pocket. (C) Rg reports compactness of the protein, ligand, and full complex. (D) The residue–ligand contact map identifies persistent and transient binding-pocket contacts over time. (E) Binding-importance ranking summarizes residues contributing most strongly to ligand engagement based on interaction persistence, frequency, type, and geometry. (F) NETWORX interaction-network visualization maps ligand atoms to interacting residues and displays the dominant interaction patterns that stabilize the binding interface.

2.4. Example use case and user-oriented workflow demonstration

To make the workflow easier for novice users to reproduce, the repository includes a complete protein-ligand use-case walkthrough based on files bundled with PyMACS: Example/CPD32_9G94.pdb, Example/A1D/A1D.cgenff.mol2, Example/A1D/A1D.str, and the final Example/MD_ANALYSIS_FIGUREBOOK.pdf. This example demonstrates the full PyMACS workflow from system preparation through molecular dynamics simulation, trajectory analysis, interaction-network rendering, and final report generation.

This section is intended to orient readers who may be familiar with molecular dynamics concepts but may not routinely write Python scripts or custom trajectory-analysis code. In practical use, PyMACS functions as a post-processing and interpretation layer after trajectory generation: the user provides the prepared structure, trajectory, topology information, ligand identifier, and a small number of analysis settings such as contact cutoff, frame interval, and output directory. PyMACS then loads the trajectory, identifies protein-ligand contacts, calculates global and ligand-specific stability metrics, ranks interacting residues, and assembles manuscript-ready figures and tables.

The analysis outputs are organized around a sequence of increasingly specific questions. First, RMSD and radius-of-gyration analyses ask whether the simulated protein-ligand complex remains structurally stable. Second, contact maps and contact-frequency outputs ask which binding-pocket residues contact the ligand consistently or transiently over time. Third, binding-importance rankings and NETWORX interaction-network diagrams translate these contacts into residue-level and atom-level summaries of the binding interface. Representative outputs from a single ligand-bound analysis run are shown in Fig. 5, while Table 3 provides a plain-language guide to the major output categories.

For routine use, the required inputs are a completed GROMACS trajectory, an associated structure file, topology or atom-selection information, the ligand identifier, and user-selected thresholds for contact detection and reporting. The generated outputs include stability plots, ligand-motion plots, compactness metrics, residue-contact heatmaps, ranked binding-residue plots, interaction-distance summaries, NETWORX network diagrams, CSV tables, and a compiled output folder suitable for manuscript figure preparation or downstream analysis.

The first stage, performed with 1_AutomateGromacs.py, guides the user through input PDB selection, ligand detection, chain naming, PDB preflight validation, ligand parameterization or CGenFF file detection, topology assembly, solvation, ionization, and generation of the energy-minimization input. A representative terminal trace of this setup workflow is shown in Supplementary Fig. S1.

The second stage, performed with 2_AutomateGromacs.py, executes energy minimization, NVT equilibration, NPT equilibration, and production molecular dynamics. In the example run, PyMACS detects the ligand A1D, configures CPU/GPU threading, generates ligand position restraints, builds the required GROMACS index groups, and completes the full simulation sequence through production MD. A complete terminal trace of this simulation stage is shown in Supplementary Fig. S2.

The third stage, performed with 3A_AutomateGromacs.py and 3B_NETWORX.py, converts the completed trajectory into analysis-ready outputs and performs structural, dynamical, and interaction-level analyses. In the example run, PyMACS writes Final_Trajectory.xtc/.pdb, extracts binding_pocket_only.xtc/.pdb, identifies the active ligand-binding chain, and computes RMSD, RMSF, Rg, secondary-structure evolution, binding-pocket contacts, interaction classifications, NETWORX panels, contact timelines, residue-level interaction summaries,

Table 3
Plain-language guide to PyMACS ligand-bound molecular dynamics analysis outputs.

Analysis stage or output	What the user provides or chooses	What PyMACS generates	Plain-language meaning	Why it matters
Trajectory setup and loading	Prepared structure, trajectory, topology or atom selections, ligand identifier, and output directory.	Validated analysis workspace with organized output folders and run-specific files.	Collects the simulation files and prepares them for analysis without requiring the user to rebuild each step manually.	Creates a reproducible starting point and reduces errors caused by inconsistent file handling.
Protein RMSD	Protein selection, trajectory, and reference structure.	Global or chain-resolved RMSD plots across the trajectory.	Shows whether the protein remains structurally stable during the simulation.	Confirms equilibration and identifies large conformational drift or unstable simulations.
Ligand RMSD	Ligand name or residue identifier and the aligned trajectory.	Ligand positional RMSD plot over time.	Shows whether the ligand remains in the original binding pose or moves within the pocket.	Helps determine whether a docked or bound pose is maintained during MD.
Radius of gyration	Protein, ligand, and complex selections.	Radius-of-gyration plot for the protein, ligand, and complex.	Shows whether the system remains compact or undergoes expansion, collapse, or large-scale breathing motion.	Provides a global structural-stability measure that complements RMSD.
Residue-ligand contact map	Contact cutoff, frame interval, ligand identifier, and optional binding-pocket filters.	Time-resolved residue-ligand contact heatmaps and contact-frequency tables.	Shows which residues contact the ligand over time and whether those contacts are persistent or transient.	Identifies residues most likely to define the stable binding interface.
Binding-importance ranking	Interaction tables and optional weighting settings for persistence, frequency, interaction type, and distance.	Ranked residue plots and tabulated binding-importance scores.	Summarizes which residues contribute most strongly to ligand engagement.	Prioritizes binding hotspots for mechanistic interpretation, mutagenesis, or ligand optimization.
NETWORX interaction network	Residue-ligand interaction data and optional display thresholds.	Ligand-centered interaction-network diagrams linking ligand atoms to interacting residues.	Converts residue contacts into a chemically interpretable map of the binding interface.	Helps users move from raw trajectory output to visual, mechanism-oriented interpretation.
Report and figure organization	Output directory and optional report-generation settings.	Compiled figure folders, CSV summaries, and report-ready visual outputs.	Collects the major stability, contact, and interaction analyses in one organized location.	Supports manuscript preparation, comparison between ligands, and transparent reporting of MD analysis results.
Comparative dashboard generation	A collection of completed PyMACS run folders organized by species, receptor, condition, or ligand state.	Static HTML dashboards, cross-system overlays, summary tables, contact/interaction matrices, stability flags, and CSV exports.	Combines many individual simulations into one interactive browser-based report.	Helps users compare drugs, receptor variants, orthologs, or species-specific behavior without manually rebuilding every plot.

Non-specialist reader note. RMSD and radius-of-gyration panels describe whether the simulated structure remains stable. Contact maps and binding-importance scores identify which residues repeatedly interact with the ligand. NETWORX converts these interactions into a visual map of the binding pocket, making it easier to interpret which chemical contacts may stabilize the ligand without requiring the reader to inspect raw trajectory files or write custom analysis code.

distance distributions, binding-importance rankings, and secondary-structure/contact-coupling outputs. A representative terminal trace of the completed analysis pipeline is shown in Supplementary Fig. S3.

The fourth stage, performed with 4PDF4MD.py, compiles the analysis outputs into the final MD_ANALYSIS_FIGUREBOOK.pdf. The user selects the ligand-bound report type, confirms the detected ligand, assigns a publication-friendly ligand name, and generates the final figurebook. A representative terminal trace of this report-generation step is shown in Supplementary Fig. S4. The resulting figurebook includes representative outputs such as global protein RMSD, ligand RMSD/RMSF, Rg, secondary-structure evolution, residue-ligand contact maps, contact-frequency rankings, interaction-type summaries, binding-importance rankings, persistence-versus-distance plots, and NETWORX interaction-network diagrams.

Additional supplementary examples extend the workflow beyond a single protein-ligand case: Supplementary Fig. S5 and S6 document cofactor-aware receptor-ligand-cofactor workflows, Supplementary Figs. S7 and S8 document RNA-containing setup and biological-mode simulation through production MD, and Supplementary Fig. S9 and S10 document PROTAC ternary complex setup and production MD, demonstrating the broad capabilities of PyMACS.

The walkthrough also documents the main user-configurable options exposed by PyMACS, including –box-type and –box-distance for simulation-box geometry, –strict-pdb-validation for input checking, –allow-covalent-ligand for expert-controlled covalent-ligand handling, –mdp-dir and stage-specific MDP overrides for simulation parameters, –index-file for custom GROMACS index groups, –ntomp, –ntmpi, –gpu-id, –gpu-ids, and –no-gpu for compute-resource allocation, –equilibration-plan for multi-stage equilibration, and –pocket-cutoff, –contact_cutoff, –hbond-distance-cutoff, and –hbond-angle-cutoff for analysis geometry. Together, Supplementary Figs. S1–S10 document representative reproducible execution paths from input structure to final analysis or production-simulation output while showing where users can inspect or modify key scientific and computational settings across protein-ligand, cofactor-aware, RNA-containing setup/biological-mode simulation examples, and PROTAC ternary complex examples.

Beyond the single-run figurebook workflow, PyMACS outputs can also be aggregated into optional browser-ready comparative dashboards. A companion 00_Pymacs_X_analysis.py workflow can scan completed run folders and generate static HTML dashboards with linked CSV exports for apo-versus-ligand, ligand-versus-ligand, receptor-family, or cross-species comparisons.

3. Discussion

3.1. Reproducibility and user-friendliness

The presented pipeline reduces MD setup to a small number of command-line arguments while retaining explicit override points for users who need to customize scientific conditions. The workflow makes default behavior and user-configurable behavior explicit: box geometry and buffer distance, MDP templates, index files, CPU/GPU resources, equilibration staging, input PDB validation, binding-pocket cutoffs, contact cutoffs, and hydrogen-bond-like classification thresholds can all be inspected or modified. By encapsulating best practices, such as fixing missing atoms, using typical CHARMM36 MD parameters with constrained hydrogen bonds, force-switching or LJ-PME nonbonded treatments, and automating CGenFF parameterization, it eliminates many common pitfalls. [13,24,25,31]. All decisions (selected force field, termini options, chain names, box settings, MDP provenance, index behavior, and execution commands) are logged, enabling other researchers to reproduce the simulation exactly. The headless mode is

particularly suited to high-throughput pipelines; the interactive mode can be used for teaching or exploratory work.

3.2. Applicability to drug discovery

The CHARMM additive force-field family covers proteins, nucleic acids, carbohydrates, lipids, and monatomic ions, and the associated CGenFF provides parameters for many drug-like small molecules. [25]. CGenFF encompasses heterocyclic scaffolds and emphasizes parameter quality over transferability [10] making PyMACS broadly relevant to medicinal chemistry and structure-based design workflows. In its current implementation, PyMACS is best viewed as an automated framework for preparing, simulating, and analyzing user-supplied candidate complexes rather than as a complete end-to-end pose-generation platform. Accordingly, the starting complex may derive from experimentally resolved structures or from upstream structure-prediction, design, and docking workflows, including AlphaFold, RoseTTAFold, Boltz-2, GNINA, Chai-1, and Protein Hunter, together with DiffDock, AutoDock, AutoDock Vina, and other related AI-based, docking-based, or physics-based methods, provided that the resulting coordinates are supplied in a compatible PDB-based format. [36–46]. Once such a starting complex is provided, PyMACS enables rapid and reproducible downstream MD setup, equilibration, production simulation, and post-simulation analysis without requiring manual intervention in the core simulation pipeline.

For RNA and other nucleic-acid-containing systems, PyMACS should be interpreted as a CHARMM/GROMACS-compatible setup and simulation framework that can be applied to compatible RNA-containing, DNA-containing, and mixed protein-nucleic-acid systems. The workflow avoids misclassifying nucleic-acid residues as ligands and can proceed when residue naming and topology generation are compatible with the installed CHARMM force-field port. Supplementary Figs. S7 and S8 provide representative evidence that a mixed RNA-protein system can be classified correctly, prepared as a biological/polymer system, equilibrated, and carried through production MD. Thus, RNA support is presented as an implemented compatible-system capability with appropriate force-field and residue-name boundaries, while broader systematic benchmarking across additional nucleic-acid target classes remains future work. These examples reconcile the RNA scope by demonstrating classification, setup, equilibration, and production simulation for a mixed RNA-protein system, while leaving systematic RNA-ligand target benchmarking and RNA-specific medicinal-chemistry validation as future work.

This design makes PyMACS immediately useful for medicinal chemistry applications in which candidate binding hypotheses are already available and require dynamic evaluation. For example, users can compare experimentally observed and computationally generated poses, assess ligand stability within a binding site, analyze residue-level interaction persistence, and examine conformational responses of the target over time. The workflow is therefore well suited for refining structure-based hypotheses, prioritizing candidate complexes for follow-up, and generating interpretable simulation outputs for protein-ligand and related ternary-complex systems. At the same time, PyMACS does not yet natively perform upstream enumeration or ranking of alternative poses, protonation states, ionization states, or tautomeric forms, which remain important considerations for broader medicinal chemistry deployment.

Similarly, the default small-molecule workflow is designed for non-covalent ligand systems. The setup script warns or stops when LINK/CONECT records suggest covalent ligand attachment, because such systems generally require explicit bonded topology definitions that are beyond the fully automated default path.

3.3. Limitations and future work

The current implementation focuses on classical MD and automated downstream simulation of user-supplied complexes. It does not currently perform free-energy calculations, enhanced-sampling methods, or explicit treatment of electronic polarization, nor does it provide an integrated upstream workflow for pose generation, multi-pose ranking, or systematic enumeration of ionization and tautomeric states. These limitations are particularly relevant in medicinal chemistry settings, where the usefulness of MD depends strongly on the quality of the starting binding hypothesis.

A key next step will therefore be to expand PyMACS into an iterative docking-to-MD workflow that more directly supports ligand-design and hypothesis-generation campaigns. In this setting, complementary docking approaches such as AutoDock and DiffDock could be used to generate candidate poses, which would then be filtered, parameterized, and advanced into standardized MD simulations within PyMACS. Such an extension would enable medicinal chemists to evaluate multiple candidate binding modes in a reproducible manner and would substantially broaden the practical utility of the framework beyond experimentally resolved complexes alone.

While CHARMM36/LJ-PME enables efficient long-range Lennard-Jones treatment and can support 9 Å nonbonded cutoffs, its benefits and parameter sensitivities should be discussed in the context of the relevant biomolecular class. Lipids required specific LJ-PME parameter optimization due to their sensitivity to Lennard-Jones truncation, whereas protein-ligand systems may require separate validation depending on the target and ligand chemistry. Additional future development should therefore expand formal benchmarking across protein-ligand, protein-protein, RNA/DNA-containing, RNA-ligand, cofactor-containing, and membrane-associated systems while building on the current RNA-containing demonstration examples.

4. Conclusion

In this work, we present a fully automated and reproducible framework for conducting molecular dynamics simulations in GROMACS using the CHARMM36 force field and CGenFF ligand parameterization. By integrating structure preparation, ligand parameterization, topology generation, solvation, equilibration, production simulation, and post-simulation analysis into a single, script-driven workflow, the pipeline substantially lowers the technical barrier for applying molecular dynamics to user-supplied biomolecular systems, including protein-ligand, cofactor-containing, protein-protein, apo, RNA-containing, DNA-containing, and mixed protein-nucleic-acid systems when force-field compatibility requirements are met, in medicinal chemistry and structure-based research.

The framework builds upon well-established and widely validated open-source tools, including the CGenFF bond-charge increment scheme for small-molecule parameterization [10] automated CHARMM-to-GROMACS conversion utilities [16] and the high-performance GROMACS simulation engine. Adherence to best practices, such as systematic energy minimization followed by sequential NVT and NPT equilibration [8] ensures numerical stability and physical realism, while support for modern force-field implementations, including CHARMM36/LJ-PME, provides accurate treatment of non-bonded interactions.

Beyond simulation execution, the pipeline places particular emphasis on standardized, high-resolution analysis and visualization, enabling quantitative assessment of structural stability, flexibility, and ligand-protein interactions in a reproducible manner. By supporting both fully automated ligand parameterization and alternative workflows that rely on pre-generated parameters, the framework remains accessible across diverse computational environments without sacrificing consistency.

The implementation distinguishes default behavior from user-configurable settings and is accompanied by repository-level walk-through, citation-inventory, and output-gallery documents to support reproducible use.

We anticipate that this level of automation and methodological transparency will facilitate broader adoption of molecular dynamics simulations within medicinal chemistry workflows, promote reproducibility across studies, and support more rigorous evaluation of biomolecular dynamics and candidate binding hypotheses in drug discovery research.

5. Availability and reproducibility

All scripts, parameter templates, documentation, and example input data used in this study are publicly available at: https://github.com/schurerlab/Pymacs. The repository contains the complete PyMACS workflow, including system preparation, molecular dynamics simulation, trajectory analysis, figure-generation scripts, external MDP templates, Conda environment definitions, use-case documentation, software/citation inventory, output-gallery documentation, and implementation notes describing repository-level changes and reproducibility features. Separate environments are provided for ligand parameterization (environment_cgenff.yml) and molecular dynamics simulation and analysis (environment_mdanalysis.yml), and a helper script (recreate_envs.sh) is included to reproduce the computational environment used in this work without reliance on container-based deployment.

To demonstrate that *PyMACS* can be applied beyond a single protein-ligand example, the packaged walkthrough focuses on the included protein-ligand example, while the manuscript and Supplementary Figs. S5–S10 document additional literature-derived, experimentally validated system classes representing distinct molecular-use cases: a conventional small-molecule protein-ligand complex [47] a receptor-ligand system containing retained cofactor/context molecules [48] an RNA-containing mixed biomolecular assembly used to demonstrate polymer-aware setup and biological-mode simulation [49] and a PROTAC-containing ternary-complex system. [50]. These examples were selected to test different branches of the *PyMACS* workflow, including standard ligand parameterization, cofactor-aware component handling, polymer-only biological-mode setup, and ternary-complex/PROTAC-aware simulation execution. The originating structures and associated molecules are drawn from previously published experimental studies, but the RNA example is presented as a representative setup/simulation demonstration rather than a complete RNA-ligand benchmarking dataset.

When SILCSBio [10,13,25] is installed locally and available in the system path, PyMACS can automatically extract and parameterize the ligand directly from the input structure during system preparation. When local SILCSBio installation is not available, the workflow remains reproducible through the use of pre-generated CGenFF output files, which PyMACS detects and incorporates automatically during topology generation. Both execution pathways produce equivalent downstream simulation behavior. All preparation, simulation, and analysis steps are fully scripted, with executed commands and intermediate outputs retained to enable complete traceability and reproducibility.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work, the author(s) used the Deep Research tool in ChatGPT (OpenAI) to assist with outlining the manuscript and generating components of figures. After using this tool/service, the author(s) reviewed and edited the content as needed and take full responsibility for the content of the published article.

Funding statement

This work was supported by the Florida Department of Health Bankhead Coley Infrastructure Grant 23B16; the Congressionally Directed Medical Research Programs (Grant1420842); and the US National Institutes of Health, including the National Cancer Institute (Grant P30 CA240139) and the National Library of Medicine (Grants U01 LM012630 and R01 LM013391). The funding agencies had no role in study design, data collection and analysis, decision to publish, or preparation of the manuscript.

CRediT authorship contribution statement

Joseph M. Schulz: Software, Validation, Writing – original draft. **Robert C. Reynolds:** Conceptualization, Writing – review & editing. **Stephan C. Schürer:** Conceptualization, Funding acquisition, Supervision, Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

We thank the Sylvester Comprehensive Cancer Center (SCCC) and the Department of Pharmacology for their continued institutional support, and the Frost Institute for Data Science & Computing (IDSC) for providing access to high-performance computing resources that enabled large-scale molecular dynamics simulations and data analysis. We also acknowledge ALAFIA for providing access to dedicated desktop computational resources used in portions of this work. We gratefully acknowledge Dr. Mainak Bardhan for his assistance with the manuscript, for testing the scripts, and for verifying that the GitHub repository and included demo function as intended.

This research also benefited substantially from the development and maintenance of open-source molecular modeling and simulation software, including GROMACS, MDAnalysis, MDTraj, PyMOL, and RDKit, as well as the CHARMM and CGenFF force-field ecosystems. We are grateful to the broader computational chemistry and structural biology communities for making these tools freely available, thereby enabling reproducible and extensible molecular dynamics research.

Appendix A. Supplementary data

Supplementary data to this article can be found online at https://doi.org/10.1016/j.ejmech.2026.119038.

Data availability

The repository for the scripts can be found here https://github.com/schurerlab/Pymacs.

References

[1] R. Sasany, A. Alizadeh, S.A. Mosaddad, P. Diaz, Molecular docking and dynamics simulation of antimicrobial peptides against adhesion proteins of peri-implant pathogens, Sci. Rep. 15 (2025) 41046.
[2] B. Raubenolt, D. Blankenberg, Generalized open-source workflows for atomistic molecular dynamics simulations of viral helicases, GigaScience 13 (2024).
[3] S.A. Hollingsworth, R.O. Dror, Molecular dynamics simulation for all, Neuron 99 (2018) 1129–1143.
[4] M. Karplus, J. Kuriyan, Molecular dynamics and protein function, Proc. Natl. Acad. Sci. U. S. A. 102 (2005) 6679–6685.
[5] D. Yu, et al., Application of the molecular dynamics simulation GROMACS in food science, Food Res. Int. 190 (2024) 114653.
[6] J.M. Schulz, S.I. Schürer, R.C. Reynolds, S.C. Schürer, PRosettaC outperforms AlphaFold3 for modeling PROTAC ternary complexes, Sci. Rep. 15 (2025) 37620.
[7] M.A. González, Force fields and molecular dynamics simulations, École thématique de la Société Française de la Neutronique 12 (2011) 169–200.
[8] J.A. Lemkul, Introductory tutorials for simulating protein dynamics with GROMACS, J. Phys. Chem. B 128 (2024) 9418–9435.
[9] A. Bunker, T. Róg, Mechanistic understanding from molecular dynamics simulation in pharmaceutical research 1: drug delivery, Front. Mol. Biosci. 7 (2020) 604770.

[10] K. Vanommeslaeghe, E.P. Raman, Jr A.D. MacKerell, Automation of the CHARMM general force field (CGenFF) II: assignment of bonded parameters and partial atomic charges, J. Chem. Inf. Model. 52 (2012) 3155–3168.
[11] O. Guvench, et al., CHARMM additive all-atom force field for carbohydrate derivatives and its utility in polysaccharide and carbohydrate-protein modeling, J. Chem. Theory Comput. 7 (2011) 3162–3180.
[12] I. Soteras Gutiérrez, et al., Parametrization of halogen bonds in the CHARMM general force field: improved treatment of ligand–protein interactions, Bioorg. Med. Chem. 24 (2016) 4812–4825.
[13] K. Vanommeslaeghe, Jr A.D. MacKerell, Automation of the CHARMM General Force Field (CGenFF) I: bond perception and atom typing, J. Chem. Inf. Model. 52 (2012) 3144–3154.
[14] Y. Yu, et al., CHARMM36 lipid force field with explicit treatment of long-range dispersion: parametrization and validation for phosphatidylethanolamine, phosphatidylglycerol, and ether lipids, J. Chem. Theory Comput. 17 (2021) 1581–1595.
[15] J. Huang, A.D. MacKerell Jr, CHARMM36 all-atom additive protein force field: validation based on comparison to NMR data, J. Comput. Chem. 34 (2013) 2135–2145.
[16] A.F. Wacha, J.A. Lemkul, Charmm2gmx: an automated method to port the CHARMM additive force field to GROMACS, J. Chem. Inf. Model. 63 (2023) 4246–4252.
[17] N.E. Jackson, B.M. Savoie, A. Statt, M.A. Webb, Introduction to machine learning for molecular simulation, J. Chem. Theory Comput. 19 (2023) 4335–4337.
[18] R. Martin-Barrios, E. Navas-Conyedo, X. Zhang, Y. Chen, J. Gulín-González, An overview about neural networks potentials in molecular dynamics simulation, Int. J. Quantum Chem. 124 (2024).
[19] Y. Wu, S. Cao, Y. Qiu, X. Huang, Tutorial on how to build Non-Markovian dynamic models from molecular dynamics simulations for studying protein conformational changes, J. Chem. Phys. 160 (2024).
[20] A.M. Brown, J.A. Lemkul, Robustness in biomolecular simulations: addressing challenges in data generation, analysis, and curation, Cell Rep. Phys. Sci. 6 (2025) 102566.
[21] A.-L.R. Brownless, E. Rheaume, K.M. Kuo, S.C.L. Kamerlin, J.C. Gumbart, Using machine learning to analyze molecular dynamics simulations of biomolecules, J. Phys. Chem. B 129 (2025) 5375–5385.
[22] H. Dong, et al., Molecular dynamics simulations of heat transport using machine-learned potentials: a mini-review and tutorial on GPUMD with neuroevolution potentials, J. Appl. Phys. 135 (2024).
[23] A. Bonkowski, R.A. De Souza, From setup to analysis: a compact guide to performing molecular dynamics simulations of ion transport in solids, Solid State Ion 429 (2025) 116967.
[24] P. Eastman, et al., OpenMM 7: rapid development of high performance algorithms for molecular dynamics, PLoS Comput. Biol. 13 (2017) e1005659.
[25] K. Vanommeslaeghe, et al., CHARMM general force field: a force field for drug-like molecules compatible with the CHARMM all-atom additive biological force fields, J. Comput. Chem. 31 (2010) 671–690.
[26] J.B. Klauda, et al., Update of the CHARMM all-atom additive force field for lipids: validation on six lipid types, J. Phys. Chem. B 114 (2010) 7830–7843.
[27] K. Hart, et al., Optimization of the CHARMM additive force field for DNA: improved treatment of the BI/BII conformational equilibrium, J. Chem. Theory Comput. 8 (2012) 348–362.
[28] E.J. Denning, U.D. Priyakumar, L. Nilsson, Jr A.D. Mackerell, Impact of 2`-hydroxyl sampling on the conformational properties of RNA: update of the CHARMM all-atom additive force field for RNA, J. Comput. Chem. 32 (2011) 1929–1943.
[29] B.R. Brooks, et al., CHARMM: the biomolecular simulation program, J. Comput. Chem. 30 (2009) 1545–1614.
[30] W. Hwang, et al., CHARMM at 45: enhancements in accessibility, functionality, and speed, J. Phys. Chem. B 128 (2024) 9976–10042.
[31] M.J. Abraham, et al., GROMACS: high performance molecular simulations through multi-level parallelism from laptops to supercomputers, SoftwareX 1–2 (2015) 19–25.
[32] M. Abraham, et al., GROMACS 2024.0 Source code, Preprint at, https://doi.org/10.5281/ZENODO.10589643, 2024.
[33] N. Michaud-Agrawal, E.J. Denning, T.B. Woolf, O. Beckstein, MDAnalysis: a toolkit for the analysis of molecular dynamics simulations, J. Comput. Chem. 32 (2011) 2319–2327.
[34] R.J. Gowers, et al., MDAnalysis: a python package for the rapid analysis of molecular dynamics simulations, SciPy (2016), https://doi.org/10.25080/Majora-629e541a-00e (2016) doi:10.25080/Majora-629e541a-00e.
[35] R.T. McGibbon, et al., MDTraj: a modern open library for the analysis of molecular dynamics trajectories, Biophys. J. 109 (2015) 1528–1532.
[36] S. Passaro, et al., Boltz-2: towards Accurate and Efficient Binding Affinity Prediction, bioRxiv, 2025 659707, https://doi.org/10.1101/2025.06.14.659707, 2025.06.14.
[37] Y. Cho, G. Rangel, G. Bhardwaj, S. Ovchinnikov, Protein hunter: exploiting structure hallucination within diffusion for protein design, bioRxiv 10 (10) (2025) 681530, https://doi.org/10.1101/2025.10.10.681530, 2025.
[38] C.D. team, et al., Chai-1: decoding the molecular interactions of life, bioRxiv 10 (10) (2024) 615955, https://doi.org/10.1101/2024.10.10.615955, 2024.
[39] G. Corso, H. Stärk, B. Jing, R. Barzilay, T. Jaakkola, DiffDock: diffusion steps, twists, and turns for molecular docking, ArXiv (2022) arXiv:2210.01776.
[40] A.T. McNutt, Y. Li, R. Meli, R. Aggarwal, D.R. Koes, GNINA 1.3: the next increment in molecular docking with deep learning, J. Cheminform. 17 (2025) 28.
[41] O. Trott, A.J. Olson, AutoDock Vina: improving the speed and accuracy of docking with a new scoring function, efficient optimization, and multithreading, J. Comput. Chem. 31 (2010) 455–461.
[42] J. Eberhardt, D. Santos-Martins, A.F. Tillack, S. Forli, AutoDock Vina 1.2.0: new docking methods, expanded force field, and Python bindings, J. Chem. Inf. Model. 61 (2021) 3891–3898.
[43] R. Evans, et al., Protein complex prediction with AlphaFold-Multimer, bioRxiv 10 (4) (2022) 463034, https://doi.org/10.1101/2021.10.04.463034, 2021.
[44] J. Abramson, et al., Accurate structure prediction of biomolecular interactions with AlphaFold 3, Nature (2024), https://doi.org/10.1038/s41586-024-07487-w, 10.1038/s41586-024-07487-w.
[45] M. Baek, et al., Accurate prediction of protein structures and interactions using a three-track neural network, Science (1979) 373 (2021) 871–876.
[46] J. Jumper, et al., Highly accurate protein structure prediction with AlphaFold, Nature 596 (2021) 583–589.
[47] T. Vantieghem, et al., Rational fragment-based design of compounds targeting the PWWP domain of the HRP family, Eur. J. Med. Chem. 280 (2024) 116960.
[48] P. Zhong, et al., Molecular basis of antagonism of the dimeric human arginine vasopressin receptor 1A, Nat. Commun. 17 (2026) 1622.
[49] C. Oubridge, N. Ito, P.R. Evans, C.H. Teo, K. Nagai, Crystal structure at 1.92 Å resolution of the RNA-binding domain of the U1A spliceosomal protein complexed with an RNA hairpin, Nature 372 (1994) 432–438.
[50] M.S. Gadd, et al., Structural basis of PROTAC cooperative recognition for selective protein degradation, Nat. Chem. Biol. 13 (2017) 514–521.